

P22 - Médian INF2 - Durée : 1h30

Documents autorisés : 1 feuille A4 recto-verso

Exercice 1 - Élections (copie n°1)

C'est le grand soir ! Tout le monde est allé voter ce dimanche et vous devez écrire un programme permettant de faciliter le dépouillement.

1.1. Préparer le dépouillement

Avant de procéder au dépouillement, il faut lister les candidats à l'élection. Disons qu'un ami s'en est chargé pour vous. De votre côté, vous avez besoin d'associer à chaque candidat un nombre de voix.

Écrire une fonction **creer_dictionnaire_candidats()** qui prend en entrée une liste des noms des candidats. Cette fonction retourne un dictionnaire dont les clés sont les noms des candidats et les valeurs le nombre de bulletins pour eux. Au début, le nombre de bulletins est à 0.

N'oubliez pas d'ajouter deux clés **Nul** et **Blanc** qui permettront de comptabiliser les bulletins nuls et blancs.

1.2. Dépouiller les bulletins

Écrire une fonction **depouiller_bulletins()** qui prend en paramètres :

- Le dictionnaire des candidats (créé précédemment).
- Une liste de bulletins (chaque bulletin est une chaîne de caractères correspondant au nom d'un candidat, ou 'Nul' ou 'Blanc').

Cette fonction mettra à jour le dictionnaire des candidats en incrémentant le nombre de voix pour chaque candidat en fonction des bulletins.

Remarque : Si un bulletin ne correspond à aucun candidat, il doit être comptabilisé comme 'Nul'.

1.3. Enregistrer les résultats

Le dépouillement est terminé ! Vous disposez donc d'un dictionnaire comptabilisant les votes pour chaque candidat ainsi que les votes blancs et nuls.

Écrire une fonction **enregistrer_resultats()** qui prend en paramètres :

- Le nom de la ville **ville**.
- Le numéro du bureau de vote **num_bureau**.
- Le dictionnaire des candidats avec leur nombre de votes associé.

Cette fonction enregistrera dans un fichier **{ville}_{num_bureau}.txt** les résultats du bureau de vote au format suivant :

Candidat 1 | 250

Candidat 2 | 3250

Candidat 3 | 890

Nul | 12

Blanc | 120

Si le fichier existe déjà, une exception sera levée pour indiquer cette anomalie et proposer une alternative.

Remarque : Pour mémoire, la fonction **os.listdir()** retourne la liste des fichiers dans le répertoire courant.

1.4. Affichage des résultats

Écrire une fonction **afficher_resultats()** qui prend en paramètre le dictionnaire des votants. Cette fonction affichera pour chaque candidat son nom et le pourcentage de votes exprimés en sa faveur.

Le pourcentage de votes en faveur d'un candidat se calcule sur le nombre de suffrages exprimés. Le pourcentage de votes nuls ou blancs se calcule sur le nombre total de votants.

Affichage attendu :

Total votants = 1000

Total exprimés = 90% (900)

Blancs = 7.5% (75)

Nuls = 2.5% (25)

Candidat 1 = 50% (450)

Candidat 2 = 30% (270)

Candidat 3 = 20% (180)

Exercice 2 - Gestion de tâches (copie n°2)

2.1. Définir la classe Task

On souhaite modéliser des tâches à effectuer. Une tâche est caractérisée par :

- Un identificateur (chaîne de caractères).
- Une durée (entier représentant le temps nécessaire pour accomplir la tâche).

Contraintes :

- Lorsqu'on essaie de définir un identificateur ou une durée qui ne respecte pas les types attendus, une exception est déclenchée.
- Lorsqu'on exécute la commande **print(t)** où **t** est un objet **Task** d'identificateur **ID** et de durée **X**, on obtient l'affichage : **ID : duree=X**.

Écrire la classe **Task** ainsi que toutes ses méthodes afin d'obtenir le comportement expliqué ci-dessus. Respecter le principe d'encapsulation.

Remarque : On remarquera qu'ici ce n'est pas la définition d'une méthode **print()** pour la classe **Task** qui est demandée (sinon, on exigerait un appel qui s'écrirait **t.print()**).

2.2. Définir la classe PriorityTask

On souhaite maintenant prendre en compte la priorité d'une tâche. Une tâche prioritaire est une tâche à laquelle on ajoute un attribut **priority** (entier entre 0 et 100).

Contraintes :

- La priorité doit être un entier entre 0 et 100. Sinon, une exception est déclenchée.
- Lorsqu'on exécute la commande **print(t)** où **t** est un objet **PriorityTask** d'identificateur **ID**, de durée **X** et de priorité **P**, on obtient l'affichage : **ID : duree=X, priorité = P**.

Écrire la classe **PriorityTask** (qui hérite de **Task**) en respectant les contraintes ci-dessus.

2.3. Gestion des contraintes de précédence

Il peut y avoir des dépendances entre certaines tâches. En particulier, on peut avoir la contrainte qu'une tâche A (de type **Task** ou **PriorityTask**) doit être terminée avant de pouvoir commencer une autre tâche B (de type **Task** ou **PriorityTask**). On appelle cela une contrainte de précédence.

Un objet de la classe **Constraint** enregistre de telles contraintes dans un attribut de type **dict** dont :

- Les clés sont des chaînes de caractères correspondant à des identificateurs de tâches.
- Les valeurs sont des listes contenant des identificateurs de tâches qui doivent être terminées avant la tâche associée à la clé.

Écrire la classe **Constraint** avec les méthodes suivantes :

- **__init__(self)** : Initialise l'attribut **dict**.
- **add_constraint(self, task_id: str, dependency_id: str)** : Ajoute une contrainte de précédence indiquant que **dependency_id** doit être terminée avant **task_id**.
- **print_constraint(self, task_id: str)** : Affiche les dépendances pour une tâche donnée, au format :

T2 -> _

T3 -> T4 T2

(Si une tâche n'a pas de dépendance, afficher _).

Utilisation de la classe Constraint

Écrire le code qui crée une instance de la classe **Constraint** et l'utilise de telle manière à provoquer l'affichage suivant :

T2 -> _

T3 -> T4 T2